

Docker Compose 기반 멀티컨테이너와 백그라운드 관리

학습 목표

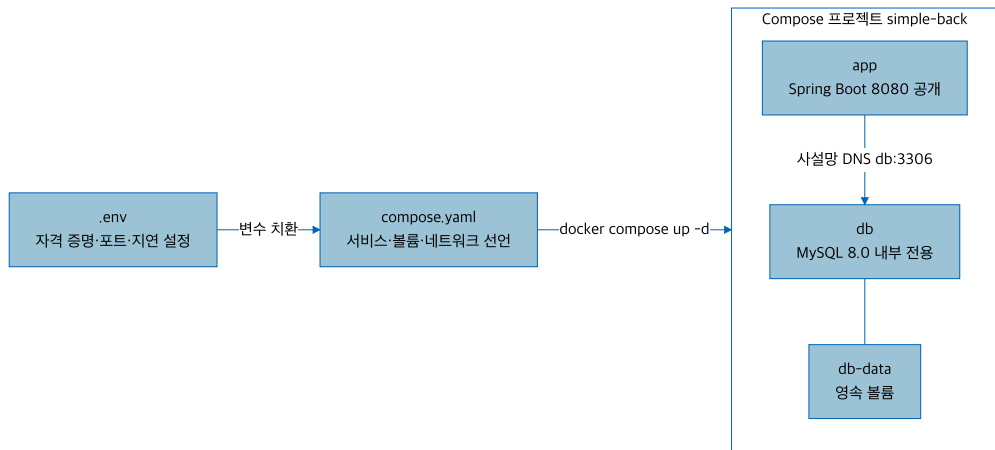
- 애플리케이션과 데이터베이스로 나뉜 2-Tier 구성을 `compose.yaml` 명세 하나로 선언합니다
- `.env` 변수 치환으로 자격 증명과 포트를 코드 밖으로 분리해 주입합니다
- `docker compose up -d`로 백그라운드 구동하고 `logs`, `top`, `exec`로 운영 상태를 점검합니다
- MySQL 초기화 지연으로 발생하는 연결 실패를 준비 확인과 재구동으로 해소합니다

컨테이너가 두 개를 넘어가면 `docker run` 명령의 나열만으로는 네트워크, 볼륨, 환경변수, 기동 순서를 매번 동일하게 재현하기 어렵습니다. 본 차시는 이 구성 전체를 **선언적 명세 파일**로 옮기고, 구동된 스택을 **백그라운드에서 관찰하고 제어하는 절차**까지 연결합니다.

핵심 개념 및 구조

2-Tier 스택의 선언과 구동 경로

설정 파일 두 개가 프로젝트 하나로 수렴하고, 그 안에서 컨테이너와 네트워크, 볼륨이 함께 생성됩니다.



단일 명세

구성이 파일 하나에 남아 그대로 재현됩니다

서비스 이름 조회

IP 대신 서비스명 `db`로 접근합니다

데이터 분리

DB 파일은 명명 볼륨에 남습니다

개별 실행 방식과 선언적 방식의 차이

비교 항목	개별 <code>docker run</code>	<code>docker compose</code>
환경 구성	네트워크, 볼륨, 컨테이너를 매번 개별 명령으로 생성	명세 한 번의 정의로 리소스 일괄 생성
서비스 조회	사용자 정의 네트워크를 수동 생성하고 플러그 주입	프로젝트 전용 사설망 자동 생성 및 이름 조회 제공
설정 관리	환경변수를 명령줄에 나열하거나 파일 옵션 지정	루트의 <code>.env</code> 를 자동 감지해 변수 치환
수명주기	컨테이너마다 <code>stop</code> , <code>rm</code> 을 순차 실행	<code>up</code> 과 <code>down</code> 으로 스택 전체를 제어

차이는 명령 길이가 아니라 **구성이 어디에 기록되는가**입니다. 개별 실행 방식에서 구성은 실행자의 터미널 기록에만 남지만, 선언적 방식에서는 파일로 남아 다른 사람과 다른 서버에서 동일하게 재현됩니다.

compose.yaml 골격과 최상위 키

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes:
      - db-data:/var/lib/mysql
    networks: [backend-net]
  app:
    build: .
    ports:
      - "${APP_PORT}:8080"
    env_file: [.env]
    depends_on: [db]
    networks: [backend-net]
volumes:
  db-data:
networks:
  backend-net:
```

services

컨테이너 단위 정의. `image`로 받아오거나 `build`로 직접 빌드합니다

volumes · networks

최상위에 선언한 이름을 각 서비스가 참조해 공유합니다

version 필드 생략

표준이 Compose Specification으로 통합되어 버전 고정이 불필요합니다

설정 주입 경로: 변수 치환과 env_file

주입 경로	적용 시점	사용처
<code>\${VAR}</code> 변수 치환	명세 파싱 시점에 문자열로 치환	이미지 태그, 포트 번호, 접속 URL 조립
<code>environment:</code>	컨테이너 생성 시 환경변수로 전달	서비스별로 값이 달라지는 항목
<code>env_file:</code>	파일의 모든 키를 일괄 전달	공통 설정을 여러 서비스에 반복 없이 주입

비밀번호를 `compose.yaml`이나 Dockerfile에 직접 적으면 저장소 이력에 평문으로 남습니다. 값은 `.env`에 두고 명세에는 변수 이름만 남기며, `.env`는 버전 관리에서 제외합니다.

구동 순서 경쟁: depends_on의 한계

시점	실제 상태	애플리케이션 측 결과
<code>depends_on</code> 조건 충족	db 컨테이너 프로세스만 시작된 상태	app 컨테이너가 곧바로 기동
MySQL 초기화 진행	테이블스페이스 생성 중, 3306 연결 거부	JPA 초기 연결 실패로 기동 중단
초기화 완료	<code>port: 3306</code> 준비 로그 출력	DB 연결 가능 상태

⌘ 연결 대기 시간

풀의 연결 획득 대기 시간을 지정해도 JPA 부팅 실패를 보장해서 막지는 못합니다

↻ 준비 확인 후 재구동

DB로그에서 준비 완료를 확인한 뒤 `docker compose restart app`으로 안착시킵니다

프로젝트 네임스페이스와 리소스 격리

리소스	명세상 이름	실제 생성 이름
네트워크	<code>backend-net</code>	<code>simple-back_backend-net</code>
볼륨	<code>db-data</code>	<code>simple-back_db-data</code>
컨테이너	<code>app</code> , <code>db</code>	프로젝트명과 서비스명 조합으로 식별

🔗 접두어의 목적

기본 프로젝트명은 디렉터리명입니다. 다른 프로젝트와 이름이 겹쳐도 리소스가 섞이지 않습니다

🖼 일괄 제어의 단위

`ps`, `logs`, `down`은 모두 이 프로젝트에 속한 리소스만 대상으로 동작합니다

실습 단계

실습 로드맵: 명세 작성부터 스택 정리까지

구간	주제	이 구간에서 직접 확인하는 것
1	프로젝트와 Dockerfile 점검	환경변수로 주입되는 DataSource 설정과 빌드 명세
2	<code>.env</code> 환경변수 중앙화	자격증명을 코드 바깥으로 분리한 단일 설정 파일
3	<code>compose.yaml</code> 서비스 선언	이미지·볼륨·네트워크·의존성의 선언적 정의
4	렌더링 검증과 일괄 구동	변수 치환 결과와 프로젝트 네임스페이스 자원
5	백그라운드 운영과 트러블슈팅	로그·프로세스·내부 명령 실행과 구동 순서 대응
6	스택 정리와 데이터 보존	컨테이너·네트워크 삭제 후에도 남는 명명 볼륨

앞 구간이 **선언**이라면 뒤 구간은 **운영**입니다. 명세 파일 하나를 작성한 뒤에는 모든 조작이 `docker compose` 하위 명령으로 수렴합니다.

환경변수 중앙화: `.env` 파일

자격증명과 포트, 연결 대기 시간을 명세 파일 바깥의 단일 설정 파일로 분리합니다.

```
DB_ROOT_PASSWORD=rootpass
DB_NAME=mydb
DB_USER=root
DB_PASSWORD=rootpass
APP_PORT=8080
SPRING_DATASOURCE_HIKARI_CONNECTION_TIMEOUT=30000
```

변수	역할
<code>DB_ROOT_PASSWORD</code> / <code>DB_PASSWORD</code>	MySQL 계정 자격증명
<code>SPRING_DATASOURCE_HIKARI_CONNECTION_TIMEOUT</code>	연결 획득 대기 시간

`.env`는 자격증명을 담으므로 **형상 관리 대상에서 제외**합니다. 저장소에는 값이 빠진 예시 파일만 두고 실제 값은 실행 환경에서 채웁니다.

서비스 명세: db와 영속 볼륨

`version:` 필드 없이 `services:`로 시작하며, 최상위 `networks` 선언이 프로젝트 전용 사설망을 만듭니다.

```
services:
  db:
    image: mysql:8.0
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - backend-net
volumes: { db-data: }
networks: { backend-net: }
```

키	역할
---	----

<code>volumes</code>	명명 볼륨 <code>db-data</code> 를 데이터 디렉터리에 연결해 컨테이너 수명과 데이터를 분리
----------------------	---

서비스 명세: `app` 과 환경변수 주입

```
app:
  build: .
  restart: on-failure
  ports:
    - "${APP_PORT}:8080"
  env_file:
    - .env
  environment:
    SPRING_DATASOURCE_URL: "jdbc:mysql://db:3306/${DB_NAME}?useSSL=false&allowPublicKeyRetrieval=true"
    SPRING_DATASOURCE_USERNAME: ${DB_USER}
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
  depends_on: [db]
  networks: [backend-net]
```

키	역할
<code>build</code>	로컬 Dockerfile을 대상으로 이미지를 직접 빌드
<code>depends_on</code>	<code>db</code> 컨테이너를 먼저 시작. 다만 DB 준비 완료까지 보장하지는 않음

기존 환경변수 수정 시 주의점

기존 `.env`의 `SPRING_DATASOURCE_HIKARI_INITIALIZATION_FAIL_TIMEOUT` 항목을 삭제합니다. 이 프로젝트에서 `-1` 주입 시 풀 설정 바인딩 실패가 관찰되었습니다.

```
docker compose up -d app
```

환경변수 변경은 컨테이너 재생성이 필요합니다. `restart app`은 기존 환경변수로 프로세스만 다시 시작합니다.

렌더링 검증과 일괄 백그라운드 구동

```
# 변수 치환과 YAML 문법 최종 확인
docker compose config --quiet

# 이미지 빌드, 네트워크·볼륨 생성, 컨테이너 일괄 실행
docker compose up -d
docker compose ps

# 프로젝트 네임스페이스가 적용된 자원 확인
docker network ls | grep backend-net
docker volume ls | grep db-data
```

명령	확인 항목
<code>docker compose config --quiet</code>	비밀 값을 출력하지 않고 종료 코드 0 확인
<code>docker network ls</code>	디렉터리명이 접두어로 붙은 <code>simple-back_backend-net</code> 확인

Compose는 네트워크와 볼륨 이름 앞에 **프로젝트 이름을 접두어로 결합**해 다른 프로젝트의 동명 자원과 충돌하지 않게 격리합니다.

백그라운드 관리 명령 한눈에 보기

명령	용도
<code>docker compose ps</code>	프로젝트에 속한 컨테이너의 상태 확인
<code>docker compose logs -f</code>	전체 서비스 로그 실시간 스트리밍
<code>docker compose logs -t --tail 20 db</code>	타임스탬프 포함, 최근 20줄만 조회
<code>docker compose top</code>	컨테이너 내부 프로세스와 호스트 PID 조회
<code>docker compose exec app printenv PORT</code>	실행 중인 컨테이너를 중단하지 않고 내부 명령 실행
<code>docker compose stop app</code> / <code>start app</code>	컨테이너를 삭제하지 않고 개별 서비스만 중지·재개

백그라운드 실행은 터미널에서 로그가 사라지는 것이지 관측 수단이 사라지는 것이 아닙니다. **상태·로그·프로세스·내부 진입** 네 가지 경로로 언제든지 확인할 수 있습니다.

MySQL 준비 완료 확인

```
# 초기화용 임시 서버(port: 0)와 구분하여 TCP 쿼리 성공을 기다린다
docker compose exec -T db sh -c '
for attempt in $(seq 1 60); do
    MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql --protocol=TCP -h127.0.0.1 \
        -uroot -Nse "SELECT 1" 2>/dev/null && exit 0
    sleep 2
done
exit 1
'
```

초기화용 임시 서버는 `port: 0`으로 표시됩니다. 최종 서버의 `port: 3306` 로그와 TCP 쿼리 성공을 함께 확인합니다.

구동 순서 문제와 준비 확인

MySQL 초기화가 끝나기 전에 JPA가 연결을 시도하면 애플리케이션이 즉시 종료됩니다.

```
# 1. DB가 쿼리를 받을 준비를 마쳤는지 먼저 확인
docker compose logs db | grep "ready for connections" |
  grep -E 'port: 3306([^\-9]|$)'
# 2. 준비 확인 후 애플리케이션만 재구동
APP_RESTART_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)
docker compose restart app
```

대응 수단	효과
TCP 쿼리와 HTTP 준비 대기	DB와 애플리케이션 준비를 각각 확인
<code>restart: on-failure</code> 와 <code>restart app</code>	준비된 DB에 커넥션 풀을 다시 안착

`depends_on`은 컨테이너 시작 순서만 보장하고 내부 서비스의 준비 완료는 보장하지 않습니다. 준비 확인은 로그나 헬스체크로 별도 수행합니다.

현재 기동의 DB 연결 확인

```
curl -fsS --retry 60 --retry-all-errors --retry-delay 2 \
  --retry-max-time 180 --max-time 3 \
  http://localhost:8080/actuator/health
docker compose logs --since "$APP_RESTART_AT" app | \
  grep "HikariPool"
curl -fsS http://localhost:8080/users
```

Up 과 Hikari **Start completed** 만으로 준비 완료를 판정하지 않습니다. 현재 기동 이후 로그와 실제 API 응답을 함께 확인합니다.

서비스 개별 제어와 엔드포인트 검증

```
# 특정 서비스만 중지 후 상태 확인
docker compose stop app
docker compose ps -a

# 서비스 재개
docker compose start app

# 웹 애플리케이션과 DB 연동 호출
curl -fsS --retry 30 \
  http://localhost:8080/actuator/health
curl -i http://localhost:8080/
curl -i http://localhost:8080/users
```

명령 / 호출	기대 결과
stop app	app은 Exited , db는 Up 유지
start app	컨테이너 재개 및 커넥션 복원
/users	HTTP 200 과 DB 조회 JSON 반환

호스트 노출 포트는 **app** 하나뿐이며, DB는 사설망 내부에서 서비스명 **db** 로만 접근됩니다.

볼륨 데이터 보존 검증

```
# 재생성 전 확인할 검증용 행 기록
docker compose exec -T db sh -c '
    MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot "$MYSQL_DATABASE"
    ' <<'SQL'
INSERT INTO users(name, email) VALUES ('Compose Check', 'compose-check@example.test')
ON DUPLICATE KEY UPDATE name = 'Compose Check';
SELECT name, email FROM users WHERE email = 'compose-check@example.test';
SQL
docker compose down
```

볼륨 목록에 이름이 남는 것에 더해, 다음 재생성 후 같은 행을 조회하여 연속성을 확인합니다.

볼륨 데이터 보존 검증

```
# 같은 프로젝트로 재생성하고 HTTP 준비 완료 후 동일 행 확인
docker compose up -d
curl -fsS --retry 60 --retry-all-errors --retry-delay 2 \
  --retry-max-time 180 --max-time 3 http://localhost:8080/actuator/health
docker compose exec -T db sh -c '
  MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot "$MYSQL_DATABASE"
  ' <<'SQL'
SELECT name, email FROM users WHERE email = 'compose-check@example.test';
SQL
```

스택 정리: `down` 과 `down -v`

명령	삭제 대상	보존 대상
<code>docker compose down</code>	컨테이너, 프로젝트 사설망	명명 볼륨 <code>simple-back_db-data</code>
<code>docker compose down -v</code>	컨테이너, 사설망, 명명 볼륨	없음. 데이터 영구 소멸

📁 기본은 볼륨 보존

일상적인 재시작과 점검에서는 `down` 만 사용합니다. 다시 `up -d` 하면 이전 데이터가 그대로 남아 있습니다.

🗑️ 스키마 초기화 전용

`-v` 는 스키마를 처음부터 다시 만들어야 하는 상황에서만 제한적으로 사용합니다.

한 장 요약

영역	핵심 요약	비고
선언적 정의	다중 컨테이너 스택을 <code>compose.yaml</code> 단일 명세로 정의	IaC 기반 재현 가능한 인프라
설정 중앙화	환경변수를 <code>.env</code> 로 격리하여 Compose 치환 및 컨테이너 주입	12-Factor App 원칙 준수
의존성·생명주기	<code>depends_on</code> 으로 기동 순서 제어, <code>restart: on-failure</code> 복구	애플리케이션 준비 완료와 분리
백그라운드 제어	<code>-d</code> 분리 실행, <code>ps</code> · <code>logs</code> · <code>down</code> 생명주기 관리	볼륨 보존(<code>down</code>) vs 볼륨 삭제(<code>down -v</code>)

Docker Compose는 애플리케이션과 데이터베이스의 네트워크, 볼륨, 환경변수를 단일 명세로 통합하여 로컬과 서버 간 일관된 환경을 보장합니다.

Appendix

서버 환경 확장: 로컬과 Ubuntu 서버

구분	로컬 개발 환경	서버 운영 환경
도구	Rancher Desktop에 엔진과 Compose 플러그인 내장	GUI 없이 APT로 <code>docker-ce</code> , <code>docker-compose-plugin</code> 설치
조작	데스크톱 UI와 터미널 병행	터미널 단독
명세 파일	<code>compose.yaml</code> 작성과 검증	동일한 파일을 수정 없이 그대로 사용

설치 방식만 다를 뿐 **표준화된 이식성** 덕분에 로컬에서 검증한 `compose.yaml` 은 서버에서도 동일하게 동작합니다. 구체적인 서버 설치 절차는 클라우드 배포 실습에서 다룹니다.

핵심 기술 면접 질문

최신 Compose의 `version` 필드 생략 이유

최신 Docker Compose에서 `version` 필드를 작성하지 않는 이유는 무엇인가요?

JPA 구동 충돌과 준비 확인 및 재구동 원리

`compose.yaml`에서 단순 `depends_on: - db`만으로 JPA 연결 실패를 완벽히 방지하지 못하는 이유와, DB 준비 확인과 재구동으로 해결하는 원리는 무엇인가요?

`top`과 `exec`의 실무 운영 활용

`docker compose top`과 `docker compose exec`는 백그라운드 컨테이너 운영 시 각각 어떤 용도로 활용되나요?

핵심 기술 면접 질문

`down` vs `down -v`의 차이와 주의점

`docker compose down`과 `docker compose down -v`의 차이점과 실무 운영 시 주의할 점은 무엇인가요?

로컬과 서버 인프라 환경 차이

Rancher Desktop에서 개발한 Docker Compose 환경을 AWS EC2 리눅스 인스턴스에 배포할 때 환경 차이와 설치 요구사항은 무엇인가요?

모범 답변

최신 Compose의 `version` 필드 생략 이유

과거에는 Docker Compose 파일 형식(v1, v2, v3)과 도커 엔진의 호환성을 맞추기 위해 파일 상단에 버전을 명시했습니다. 하지만 오픈소스 Compose Specification으로 표준이 통합되면서, 최신 Compose 엔진은 버전을 고정하지 않고 최신 스펙을 자동으로 해석합니다. 따라서 현재는 불필요한 `version` 필드를 생략하는 것이 공식 표준입니다.

JPA 구동 충돌과 준비 확인 및 재구동 원리

기본 `depends_on`은 컨테이너 시작 순서만 제어하며 MySQL의 준비 완료는 보장하지 않는다. JPA가 준비 전 DB에 연결하면 기동이 실패할 수 있다. MySQL 최종 서버의 `port: 3306` 로그와 TCP 쿼리 성공을 확인한 뒤 `docker compose restart app`으로 재구동하고, 제한 시간 내 HTTP 준비 대기과 `/users` 응답으로 연결을 검증한다. 환경변수를 변경한 경우에는 `docker compose up -d app`으로 재생성한다.

`top` 과 `exec` 의 실무 운영 활용

`docker compose top`은 Compose가 관리하는 모든 백그라운드 컨테이너 내부 프로세스와 호스트 OS의 실제 PID를 한눈에 모니터링하여 자원 점유를 확인할 때 사용합니다. `docker compose exec`는 구동 중인 서비스 컨테이너를 재시작하지 않고 내부에 진입하여 환경변수(`env`), 설정 파일, DB CLI 쿼리 등을 즉각 디버깅할 때 활용하는 필수 운영 명령어입니다.

모범 답변

`down` vs `down -v`의 차이와 주의점

`docker compose down`은 컨테이너와 가상 네트워크만 중지하고 삭제하므로, 명명된 볼륨에 저장된 데이터베이스 데이터는 영구히 유지됩니다. 반면 `-v` 옵션을 붙이면 볼륨까지 강제로 삭제되므로 DB의 모든 데이터가 영구 유실됩니다. 운영 환경에서는 실수로 데이터가 삭제되는 사고를 방지하기 위해 `-v` 플래그 사용을 엄격히 금지해야 합니다.

로컬과 서버 인프라 환경 차이

로컬 머신에서는 Rancher Desktop에 Docker 엔진과 Compose 플러그인이 내장되어 즉시 사용할 수 있습니다. 반면 클라우드 운영 환경인 AWS EC2는 순수 리눅스(Ubuntu) 가상 서버이므로 GUI 없이 터미널에서 패키지 관리자(APT)를 통해 `docker-ce` 및 `docker-compose-plugin`을 직접 설치해야 합니다. 도구 설치 방식만 다를 뿐, 도커의 표준화된 이식성 덕분에 로컬에서 검증한 `compose.yaml`은 EC2에서도 코드 수정 없이 100% 동일하게 동작합니다.

깊이 알아보기

Docker Compose 개요 Docker Compose

멀티 컨테이너 정의와 실행 공식 가이드

<https://docs.docker.com/compose/>

Compose 파일 사양 Compose file reference

services·volumes·networks 키 전체 사양

<https://docs.docker.com/reference/compose-file/>

CLI 하위 명령 docker compose

up·ps·logs·top·exec·down 옵션 원문

<https://docs.docker.com/reference/cli/docker/compose/>

변수 치환 Variable interpolation

환경변수 파일 적용 순서와 치환 규칙

<https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/>

깊이 알아보기

구동 순서 제어 Control startup order

의존성 선언과 준비 상태 확인 방법

<https://docs.docker.com/compose/how-tos/startup-order/>

Compose Specification compose-spec

버전 필드 없는 표준 사양 원문

<https://github.com/compose-spec/compose-spec/blob/main/spec.md>



- EOF -